DALHOUSIE UNIVERSITY

Cleaning and Transforming Data

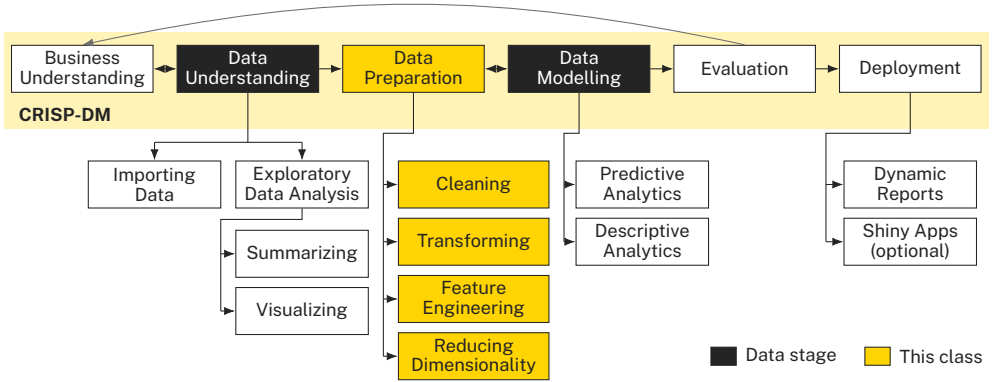
Feature engineering, data reduction, category coding, and monitoring records

Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Prepare a Table You Can Explain

- Create a derived variable with explicit units.
- Reduce a dataset by sampling, and code categories as indicators.
- Preserve raw monitoring records and document each cleaning decision.

Data Preparation in the Course Workflow



Summaries and plots check preparation decisions. The workflow can loop back.

Build a Variable with Meaningful Units

A derived variable combines existing measurements.

$$\text{BMI} = \frac{\text{mass in kg}}{(\text{height in m})^2}$$

For 82 kg and 178 cm, first convert the height to metres.

$$82/1.78^2 = 25.88 \text{ kg/m}^2$$

The unit conversion is part of the calculation. BMI is used here to illustrate arithmetic, not to make a clinical judgement.

Keep the Source Columns

R

```r
measurements <- data.frame(
  mass_kg = c(82, 61, 95),
  height_cm = c(178, 165, 181))
measurements$bmi <- measurements$mass_kg /
  (measurements$height_cm / 100)^2
round(measurements$bmi, 2)
# 25.88 22.41 29.00
```

Python

```python
import pandas as pd

measurements = pd.DataFrame({
    "mass_kg": [82, 61, 95],
    "height_cm": [178, 165, 181]})
measurements["bmi"] = (
    measurements["mass_kg"]
    / (measurements["height_cm"] / 100)**2)
print(measurements["bmi"].round(2).to_list())
# [25.88, 22.41, 29.0]
```

Columns from the same table refer to the same rows. R vector arithmetic uses position; pandas Series arithmetic aligns index labels.

Three Different Things You Can Reduce

DALHOUSIE UNIVERSITY

Fewer variables	Fewer observations	Fewer distinct values
Keep only the variables the question needs	Random sample Stratified sample	Bin numeric values Combine categories only with a clear reason

Each reduction discards something. Choose it for an analytical reason, not because the resulting file is smaller.

Random and Stratified Samples

R

```r
data <- data.frame(row_id = 1:10,
  site = rep(c("A", "B"), each = 5))
set.seed(3141)
idx <- sample(seq_len(nrow(data)), size = 6)
random_sample <- data[idx, ]
# Draw three rows from each site.
idx_by_site <- split(
  seq_len(nrow(data)), data$site)
chosen <- unlist(
  lapply(idx_by_site, sample, size = 3))
stratified_sample <- data[chosen, ]
table(stratified_sample$site)
# A 3, B 3
```

Python

```python
import pandas as pd

data = pd.DataFrame({
    "row_id": range(1, 11),
    "site": ["A"]*5 + ["B"]*5})
random_sample = data.sample(
    n=6, random_state=3141)
# Draw three rows from each site.
stratified_sample = data.groupby("site").sample(
    n=3, random_state=3141)
print(stratified_sample["site"].value_counts())
# A 3, B 3
```

Both samples contain six rows. Stratification fixes three per site. Equal seeds do not imply identical R and Python samples.

Encode Categories, Not an Order

R

```r
site <- factor(c("A", "B", "A", "C"))
indicators <- model.matrix(~ site - 1)
as.data.frame(indicators)
# Three columns, one per site.
```

Python

```python
import pandas as pd

site = pd.Series(["A", "B", "A", "C"])
indicators = pd.get_dummies(site, dtype=int)
print(indicators)
# Three columns, one per site.
```

Each row has one indicator equal to 1. Coding sites A, B, C as 1, 2, 3 would suggest an order and equal spacing that these labels do not have. With C categories, a regression keeps $C - 1$ indicators and drops one level.

Four Different Value Statuses

Raw field	Status	Interpretation
`0`	measured	Exact recorded zero in the stated unit.
blank	missing	No usable measurement was recorded.
`<0.1`	censored	Below 0.1 mg/L; retain 0.1 as a limit, not a value.
`oops`	invalid	Unexpected token; investigate the original record.
`0.24`	measured	Exact recorded value, subject to provider quality flags.

Readings are in mg/L. A reporting limit bounds an unobserved measurement; it is not the measurement itself. Keep quality flags and check the provider's definitions.

Preserve the Record, Add an Interpretation

R

```r
raw <- data.frame(
  value_text = c("0", "", "<0.1", "oops",
                 "0.24"),
  unit = rep("mg/L", 5),
  stringsAsFactors = FALSE)
cleaned <- raw
cleaned$status <- c(
  "measured", "missing", "censored",
  "invalid", "measured")
cleaned$value_mg_L <- c(0, NA, NA, NA, 0.24)
cleaned$limit_mg_L <- c(NA, NA, 0.1, NA, NA)
stopifnot(identical(
  raw$value_text, cleaned$value_text))
# These statuses follow the stated rules.
# Original strings and units remain.
```

Python

```python
import pandas as pd

raw = pd.DataFrame({
    "value_text":
        ["0", "", "<0.1", "oops", "0.24"],
    "unit": ["mg/L"]*5})
cleaned = raw.copy()
cleaned["status"] = [
    "measured", "missing", "censored",
    "invalid", "measured"]
cleaned["value_mg_L"] = [
    0, None, None, None, 0.24]
cleaned["limit_mg_L"] = [
    None, None, 0.1, None, None]
assert raw["value_text"].equals(
    cleaned["value_text"])
```

Validated Conversion, Classify Each Token

R

```r
text <- trimws(raw$value_text)
# This provider defines blank as missing.
missing <- text == ""
censored <- grepl("^<[0-9]+([.][0-9]+)?$", text)
measured <- grepl("^[0-9]+([.][0-9]+)?$", text)
cleaned$status <- ifelse(missing, "missing",
  ifelse(censored, "censored",
    ifelse(measured, "measured", "invalid")))
```

Python

```python
text = raw["value_text"].str.strip()
# This provider defines blank as missing.
missing = text.eq("")
censored = text.str.fullmatch(
    r"<[0-9]+([.][0-9]+)?")
measured = text.str.fullmatch(
    r"[0-9]+([.][0-9]+)?")
cleaned["status"] = "invalid"
for mask, label in [(missing, "missing"),
                    (censored, "censored"),
                    (measured, "measured")]:
    cleaned.loc[mask, "status"] = label
```

Continue from the raw and cleaned copies. These example rules recognize whole unsigned decimals and less-than limits; unrecognized tokens stay invalid.

Validated Conversion, Convert the Values

R

```r
cleaned$value_mg_L <- NA_real_
cleaned$value_mg_L[measured] <-
  as.numeric(text[measured])
cleaned$limit_mg_L <- NA_real_
cleaned$limit_mg_L[censored] <-
  as.numeric(sub("^<", "", text[censored]))
cleaned$value_ug_L <- 1000 * cleaned$value_mg_L
cleaned
```

Python

```python
cleaned["value_mg_L"] = pd.to_numeric(
    text.where(measured))
cleaned["limit_mg_L"] = pd.to_numeric(
    text.where(censored).str.removeprefix("<"))
cleaned["value_ug_L"] = (
    1000 * cleaned["value_mg_L"])
print(cleaned)
```

Only measured tokens get a value and only censored tokens get a limit; every other row stays missing. The notes explain each pattern and give the full script.

Count Repeated Keys Before Removing Rows

R

```r
d <- data.frame(
  station = c("S1", "S1", "S1", "S2"),
  date = c("2026-09-01", "2026-09-01",
           "2026-09-02", "2026-09-01"),
  parameter = rep("nitrate", 4),
  value_mg_L = c(0.24, 0.31, 0.22, 0.18)
)
key <- c("station", "date", "parameter")
repeated <- duplicated(d[key]) |
  duplicated(d[key], fromLast = TRUE)
sum(repeated)   # 2
d[repeated, ]
#   station       date parameter value_mg_L
# 1      S1 2026-09-01   nitrate       0.24
# 2      S1 2026-09-01   nitrate       0.31
```

Python

```python
import pandas as pd

d = pd.DataFrame({
    "station": ["S1", "S1", "S1", "S2"],
    "date": ["2026-09-01", "2026-09-01",
             "2026-09-02", "2026-09-01"],
    "parameter": ["nitrate"] * 4,
    "value_mg_L": [0.24, 0.31, 0.22, 0.18],
})
key = ["station", "date", "parameter"]
repeated = d.duplicated(subset=key, keep=False)
print(repeated.sum())   # 2
print(d[repeated])
#   station        date parameter  value_mg_L
# 0      S1  2026-09-01   nitrate        0.24
# 1      S1  2026-09-01   nitrate        0.31
```

Both S1 rows dated 2026-09-01 are marked and kept; their values differ.

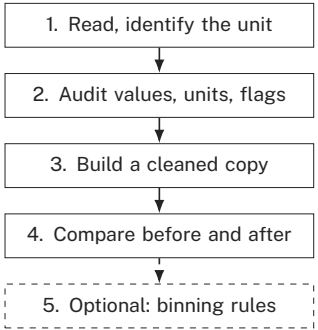
Check Meaning Before Arithmetic

- Keep the downloaded file unchanged and preserve original strings.
- Identify station, time, parameter, unit, and quality flags; count repeated keys.
- Separate censored limits from exact measurements; investigate invalid tokens.
- Convert compatible units only with a stated rule, such as $1\,\mathrm{mg/L} = 1000\,\mu\mathrm{g/L}$.
- Report missing and censored counts beside summaries of usable measurements.

Flagging an unusual reading is a reason to investigate. It is not evidence that the reading should be deleted.

One Record, a Documented Cleaning Path

1. Read one approved monitoring file and identify its observational unit.
2. Audit missing values, reporting limits, types, units and quality flags.
3. Build a cleaned copy with a reason for each changed or excluded value.
4. Compare summaries before and after cleaning, keeping the raw file unchanged.
5. If categories help the question, compare two binning rules on one suitable numeric variable.



What to Take Away

- A derived variable is only as correct as its units. Convert before you compute.
- Every reduction discards something. Stratified sampling keeps each group represented.
- Indicators record membership without inventing an order.
- Measured, missing, censored, and invalid are different statuses. Keep the raw string and report every count.
- Count repeated keys before removing any row.

References

[1] D. R. Helsel. “Fabricating data: how substituting values for nondetects can ruin results, and what can be done about it”. In: *Chemosphere* 65.11 (2006), pp. 2434–2439.